

Hydrogenic-demo

Hydrogenic-demo provides a compact C++ example for hydrogenic atom calculations, together with a Python interface for interactive work and notebook-based demonstrations. The C++ code builds the `hydrogenic_atom_demo` executable and exercises the routines in `Hydrogenic.vX` together with numerical utilities from `Tools`.

Project layout

Path	Purpose
<code>main.cpp</code>	Example program that constructs hydrogenic atoms and prints representative atomic data.
<code>Hydrogenic.vX/Atom.*</code>	Core <code>Atom</code> , <code>Atomic_Shell</code> , <code>Electron_Level</code> , and <code>Gas_of_Atoms</code> classes.
<code>Hydrogenic.vX/Oscillator_strength.*</code>	Bound-bound oscillator strength and transition support.
<code>Hydrogenic.vX/Photoionization_cross_section.*</code>	Photoionization cross sections and Gaunt factors.
<code>Hydrogenic.vX/Quadrupole_lines_HI.*</code>	Electric quadrupole line rates for hydrogenic atoms.
<code>Hydrogenic.vX/Rec_Phot_BB.*</code> <code>Recombination_Integrals.*</code>	and Recombination and photon interaction helpers.
<code>Tools/</code>	Shared constants, integration routines, line profiles, and simple numerical helpers.
<code>python/</code>	Installable Python package and Jupyter notebook examples.
<code>Documentation/</code>	PDF documentation and editable TeX source.
<code>Makefile</code> and <code>Makefile.in</code>	Build entry points for the C++ demo executable.

What the C++ example does

The demo constructs hydrogenic atoms with a selected number of complete shells. Each shell contains angular momentum levels with $l = 0 \dots n - 1$. The code can report:

- level degeneracies, ionization energies, ionization frequencies, total decay rates, and lifetimes;
- selected bound-bound transition frequencies, wavelengths, and Einstein A21 coefficients;
- electric dipole transitions and optional electric quadrupole transitions;
- photoionization cross sections and Gaunt factors at frequencies above threshold;
- simple rescaling behavior for the fine-structure constant and electron mass.

The current `main.cpp` includes two examples:

- hydrogen with $Z = 1$, 5 shells, quadrupole lines enabled, and Storey-Hummer recombination interpolation enabled;
- hydrogenic oxygen, O VIII, with $Z = 8$, 30 shells, quadrupole lines enabled, and Storey-Hummer recombination interpolation enabled.

Scientific background

The hydrogenic atom setup was developed for detailed calculations of cosmological recombination. This requires multi-shell hydrogenic atoms with resolved angular momentum sub-states, dipole transitions, quadrupole transitions, photoionization cross sections, and recombination coefficients. The C++ example in `main.cpp` cites the main background papers for this use case.

Dependencies

The C++ build expects:

- a C++17 compiler with OpenMP support;
- GSL headers and libraries;
- Boost headers;
- GNU make.

The Python package additionally uses:

- Python 3.9 or newer;
- pybind11 and numpy at build time;
- optional jupyter and matplotlib for notebooks and plots.

On macOS with Homebrew, the typical compiled dependency setup is:

```
brew install gcc gsl boost
```

`Makefile.in` currently defaults to:

```
CXX = /opt/homebrew/bin/g++-16
DEF_INC_PATH = /opt/homebrew/include
DEF_LIB_PATH = /opt/homebrew/lib
```

If your compiler or Homebrew prefix differs, override these variables when invoking `make` or `pip`.

Build and run the C++ demo

From the workspace root:

```
make
```

List available Makefile targets:

```
make help
```

Aliases are also available:

```
make h
make usage
make list
make targets
```

`make --help` and `make -h` are handled by GNU make itself; use `make help` for project-specific help.

Run the demo:

```
make run
```

Clean generated build products:

```
make clean
```

Update submodules if the project is checked out with submodule dependencies:

```
make update-modules
```

Example build override:

```
make CXX=/opt/homebrew/bin/g++-16 DEF_INC_PATH=/opt/homebrew/include  
DEF_LIB_PATH=/opt/homebrew/lib
```

The executable is written to:

```
./hydrogenic_atom_demo
```

Intermediate C++ object and dependency files are written under:

```
./build/
```

Python extension build products are written under `python/build/` and `python/src/hydrogenic/`. `make clean` removes both the C++ and Python build products.

Python package and Jupyter

Use `python3`, not `python`, from the repository root. Since this repository has a `python/` folder, some shell setups can otherwise resolve `python` incorrectly and report `Permission denied`.

To build the Python extension and run the notebook with existing Homebrew/JupyterLab installations, you need:

- `numpy`;
- `matplotlib`;
- `jupyterlab` or `notebook`;
- build-time `setuptools` and `pybind11`;
- compiled dependencies: GCC with OpenMP, GSL, and Boost headers.

On macOS with Homebrew:

```
brew install gcc gsl boost python-setuptools pybind11 numpy matplotlib jupyterlab
```

Build the local Python extension:

```
make python-build  
make python  
make py
```

Open the demo notebook:

```
make run-jupyter
```

make run-jupyter runs make python-build first, then opens the notebook with an available Jupyter command (jupyter lab, jupyter-lab, or jupyter-notebook) and sets PYTHONPATH to include python/src. This lets the notebook import the local hydrogenic extension without installing it into JupyterLab's private Python environment.

The Makefile workflow avoids installing the package into your system Python. If you deliberately want a pip install into a Homebrew-managed Python, Homebrew may require:

```
python3 -m pip install --user --break-system-packages ./python
```

If needed, override the compiler and dependency paths explicitly:

```
CXX=/opt/homebrew/bin/g++-16 \  
GSL_INC_PATH=/opt/homebrew/include \  
GSL_LIB_PATH=/opt/homebrew/lib \  
BOOST_INC_PATH=/opt/homebrew/include \  
python3 -m pip install ./python
```

Use from Python

```
import numpy as np  
from hydrogenic import hydrogen, oxygen  
  
h = hydrogen(shells=5)  
print(h.level(2, 1))  
print(h.transition(2, 1, 1, 0))  
  
o = oxygen(shells=30)  
nu0 = o.level(1, 0)["ionization_frequency_hz"]  
nu = np.geomspace(1.001, 10.0, 100)*nu0  
data = o.photoionization(1, 0, nu)  
print(data["sigma_cm2"])
```

Photoionization cross sections require recombination_mode=1 or recombination_mode=2; this is the default used by the Python convenience constructors.

Python notebooks

The notebook examples live in python/notebooks/. They illustrate the same routines from Python, including level data, line transitions, and photoionization cross-section plots for hydrogen and hydrogenic oxygen.

Documentation files

The PDF documentation is:

[Documentation/Hydrogenic_cpp_Documentation.pdf](#)

The editable TeX source is:

[Documentation/Hydrogenic_cpp_Documentation.tex](#)

The PDF can be regenerated from this README with Pandoc and Typst:

```
pandoc README.md --from gfm --pdf-engine=typst --output Documentation/  
Hydrogenic_cpp_Documentation.pdf
```

If a LaTeX installation is available, the TeX source can be edited and compiled by hand:

```
cd Documentation  
pdflatex Hydrogenic_cpp_Documentation.tex
```

References

- [1] J. Chluba and R. M. Thomas, "Towards a complete treatment of the cosmological recombination problem", MNRAS 412, 748, 2011. <https://ui.adsabs.harvard.edu/abs/2011MNRAS.412..748C/abstract>
- [2] J. Chluba, J. A. Rubino-Martin, and R. A. Sunyaev, "Cosmological hydrogen recombination: populations of the high level sub-states", MNRAS 374, 1310, 2007. <https://ui.adsabs.harvard.edu/abs/2007MNRAS.374.1310C/abstract>
- [3] J. Chluba and Y. Ali-Haïmoud, "CosmoSpec: Fast and detailed computation of the cosmological recombination radiation from hydrogen and helium", MNRAS 456, 3494, 2016. <https://ui.adsabs.harvard.edu/abs/2016MNRAS.456.3494C/abstract>